Procedure/Program:

1. Discretionary Access Control (DAC)

a. Create Files and Directories:

Command

```
mkdir dac_demo
cd dac_demo
touch confidential.txt
```

What is happening?

mkdir dac_demo: Creates a directory named dac_demo.

touch confidential.txt: Creates an empty file named confidential.txt inside the directory.

b. Set Permissions Using chmod:

Command:

```
chmod 600 confidential.txt
```

The chmod 600 command changes the file's permissions:

Owner: Can read and write.

and Others: No permissions.

c. Change File Ownership Using chown:

Commands:

```
sudo adduser alice
sudo chown alice:alice confidential.txt
```

What is happening?

sudo adduser alice: Creates a new user named Alice.

sudo chown alice:alice confidential.txt: Transfers ownership of the file confidential.txt to Alice.

20

d. Log in as Alice:

Command:

```
su alice
cat confidential.txt
```

What is happening?

su alice: Switches to the user Alice.

cat confidential.txt: Tries to read the file.

e. Results:

Alice, as the file owner, can access confidential.txt.

Other users (e.g., the default user) cannot read or write to the file because of the restrictive permissions (600).

```
atharv@atharv:~/Documents/IS$ mkdir dac_demo
atharv@atharv:~/Documents/IS$ cd dac_demo/
atharv@atharv:~/Documents/IS/dac_demo$ touch confidential.txt
atharv@atharv:~/Documents/IS/dac_demo$ chmod 600 confidential.txt
atharv@atharv:~/Documents/IS/dac_demo$ sudo adduser alice
[sudo] password for atharv:
sudo: a password is required
atharv@atharv:~/Documents/IS/dac_demo$ sudo adduser work
[sudo] password for atharv:
adduser: The user `work' already exists.
atharv@atharv:~/Documents/IS/dac_demo$ sudo adduser alice
Adding user `alice' ...
Adding new group `alice' (1002) ...
Adding new user `alice' (1002) with group `alice' ...
Creating home directory `/home/alice' ...
Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
Changing the user information for alice
Enter the new value, or press ENTER for the default
        Full Name []: Atharv
        Room Number []: 1
        Work Phone []: 1
        Home Phone []: 1
        Other []: 1
Is the information correct? [Y/n] Y
atharv@atharv:~/Documents/IS/dac_demo$ sudo chown alice:alice confidential.txt
atharv@atharv:~/Documents/IS/dac_demo$ su alice
Password:
alice@atharv:/home/atharv/Documents/IS/dac_demo$ cat confidential.txt
alice@atharv:/home/atharv/Documents/IS/dac_demo$
```

2. Mandatory Access Control (MAC)

a. Enable SELinux:
Command : getenforce
What is happening?
Checks the status of SELinux (enforcing, permissive, or disabled).
If SELinux is not enabled:
sudo setenforce 1
This activates SELinux in "enforcing" mode.

b. Apply Security Context to a File:
Command: ls -Z confidential.txt
What is happening?
The ls -Z command displays the SELinux context of the file.
Example output: user_u:object_r:default_t:s0
confidential.txt
Change the context to allow access by specific processes:
sudo chcon -t httpd_sys_content_t confidential.txt
What is happening?
Changes the file's security context so that only the httpd
(web server) process can access it.

c. Test Policy Enforcement:
Try accessing the file through an unauthorized process.
Logs: sudo cat /var/log/audit/audit.log
What is happening?
SELinux denies access to unauthorized processes and logs
the event in audit.log.20

d. Results:
Access is granted or denied strictly based on SELinux policies,not ownership or file permissions.
Unauthorized processes will be denied access, even if they are run by the file owner.

```
atharv@atharv:~/Documents/IS$ getenforce
Disabled
atharv@atharv:~/Documents/IS$ sudo setenforce 1
setenforce: SELinux is disabled
atharv@atharv:~/Documents/IS$ ls
dac_demo  sensitive_file.txt
atharv@atharv:~/Documents/IS$ cd dac_demo/
atharv@atharv:~/Documents/IS/dac_demo$ ls
confidential.txt
atharv@atharv:~/Documents/IS/dac_demo$ ls -Z confidential.txt
? confidential.txt
atharv@atharv:~/Documents/IS/dac_demo$ sudo cat /var/log/audit/audit.log
cat: /var/log/audit/audit.log: No such file or directory
atharv@atharv:~/Documents/IS/dac_demo$
```

3. Role-Based Access Control (RBAC)

Create Roles (Groups):
Commands:
sudo groupadd managers
sudo usermod -aG managers alice
What is happening?
sudo groupadd managers: Creates a group called managers.
sudo usermod -aG managers alice: Adds Alice to the managers group.

Set Permissions on a File:
Commands:
touch manager_notes.txt
sudo chown :managers manager_notes.txt
sudo chmod 770 manager_notes.txt
What is happening?
touch manager_notes.txt: Creates a new file named manager_notes.txt.
o sudo chown :managers manager_notes.txt: Changes the file's group ownership to managers.
sudo chmod 770 manager_notes.txt: Sets permissions:
Owner and group: Read, write, and execute.
Others: No permissions.

Test Access:
As Alice (a member of the managers group):
su alice
cat manager_notes.txt
As another user (not in the managers group):
su <other_user>
cat manager_notes.txt
What is happening?
Alice can access the file because she belongs to the managers group.
Other users are denied access.

Results:

Access to manager_notes.txt is controlled by group membership (role), not individual ownership.

This approach simplifies access control in organizations where users' roles define their permissions.

```
atharv@atharv:~/Documents/IS/dac_demo$ sudo groupadd managers
atharv@atharv:~/Documents/IS/dac_demo$ sudo usermod -aG managers alice
atharv@atharv:~/Documents/IS/dac_demo$ touch manager_notes.txt
atharv@atharv:~/Documents/IS/dac_demo$ sudo chown :managers manager_notes.txt
atharv@atharv:~/Documents/IS/dac_demo$ sudo chmod 770 manager_notes.txt
atharv@atharv:~/Documents/IS/dac_demo$ su alice
Password:
alice@atharv:/home/atharv/Documents/IS/dac_demo$ cat manager_notes.txt
alice@atharv:/home/atharv/Documents/IS/dac_demo$ su work
Password:
work@atharv:/home/atharv/Documents/IS/dac_demo$ cat manager_notes.txt
cat: manager_notes.txt: Permission denied
work@atharv:/home/atharv/Documents/IS/dac_demo$
```